

BDF - Customized Product Documentation

--- bdf_bakery_product_doc.md ---

<p align="center">

</p>

BDF Bakery Production Monitoring System - Product Documentation

This document provides essential information about the Racer2 V3 system, designed to enhance BDF's bakery automation and production monitoring. It explains how the system operates, how to configure it, and how to troubleshoot common issues, all with a focus on operator-friendly explanations and maintaining high-quality bakery product output.

1. System Overview

The Racer2 V3 is a powerful AI Core service specifically developed for BDF's line-scan acquisition, image stitching, live streaming, and optional AI-assisted inspection of bakery products. It's built to work seamlessly with two Basler line cameras, offering both real-hardware and hardware-free development options to suit various operational needs at BDF's facilities.

1.1. Purpose for BDF

The primary goal of the Racer2 V3 system is to provide BDF with advanced tools for:

- **Automated Quality Control:** Continuously monitor bakery products on the line for consistent quality.
- **Production Monitoring:** Gain real-time insights into the production process, identifying any deviations quickly.
- **Operational Efficiency:** Streamline inspection processes through automation, reducing manual effort.
- **Data-Driven Decisions:** Collect valuable data on product characteristics, including color, for analysis and improvement.

1.2. Key Capabilities

The BDF Bakery Production Monitoring System offers:

- **Dual-Camera Acquisition:** Captures high-resolution images from two cameras simultaneously.

- **Stitched Image Generation:** Combines individual camera images into a single, comprehensive view of the bakery product.
- **Live Production Stream:** Provides browser-friendly JPEG images and live WebSocket streaming for immediate visual feedback.
- **Flexible Camera Access:** Supports direct camera connection, remote monitoring, or simulated frame generation for testing.
- **Optional AI Inspection:** Integrates advanced AI for automated defect detection and product tracking.
- **Color Analysis:** Tracks and analyzes the color of bakery products, buffering recent history for quality checks.
- **External System Integration:** Allows forwarding of production images for further analysis by other systems.
- **Local Analysis:** Supports file-based local AI analysis of recorded images.

1.3. How It Operates

The system can run in three distinct camera modes to match BDF's operational setup:

- **Local Mode:** The main system directly controls the Basler cameras, ideal for an all-in-one setup on the production floor.
- **Remote Mode:** The main system fetches images from a separate camera service, allowing camera hardware to remain close to the production line while the monitoring interface can be located elsewhere.
- **Emulated Mode:** The system generates simulated frames, perfect for testing the user interface and API functionalities without needing physical cameras, preventing disruption to live production.

AI processing can be configured to:

- **Disabled:** Provides raw production imagery for visual inspection only.
- **Enabled with RF-DETR:** Utilizes advanced detection and tracking for precise quality control.
- **Enabled with YOLO and SORT:** Offers an alternative AI approach for detection and tracking.

1.4. High-Level Process Flow

1. The main application starts, initializing the core system.
2. A camera runtime is selected based on the configured mode (local, remote, or emulated).
3. The system begins capturing, polling, or generating JPEG images of your bakery products.
4. If AI is enabled, the AI models start in the background, reporting their progress.
5. The latest image of the bakery product is made available through various access points:
 - `GET /capture``: For retrieving the most recent image.
 - `POST /analyze``: For sending images to external quality inspection services.

- ``POST /inference``: For performing local AI analysis on specific images.
- ``WebSocket /ws``: For a live stream to operator dashboards.

6. If color analysis is active, tracked object samples are collected and available through dedicated color data endpoints.

[_Space for diagram: High-Level System Flow_]

1.5. Core System Components

The Racer2 V3 system consists of several key software components:

- **Server Layer**: Manages the application's overall behavior, endpoints, camera mode selection, and AI status reporting. This is where operators interact with the system.
- **User Interface (UI)**: Provides a browser-based viewer and health dashboard for operators to monitor production.
- **Camera Library**: Handles the technical aspects of camera operation, image acquisition, stitching, resizing, and JPEG encoding for live production monitoring.
- **AI Library**: Manages AI model selection, analysis of live and stored images, defect tracking, and visual annotation overlays.
- **Color Library**: Focuses on extracting and buffering color data for each tracked bakery product, enabling detailed color quality analysis.
- **Documentation**: This technical documentation and operational guides.

1.6. Service Separation

For enhanced flexibility, BDF's Racer2 V3 can run as a single integrated process or as two independent services:

- **Main Application**: Running on port ``8000``, providing the UI, AI, and overall monitoring.
- **Dedicated Camera Service (Optional)**: Running on port ``8001``, specifically handling camera interaction.

This separation is beneficial when the camera hardware needs to remain on the production line, while the web interface or AI processing can be hosted on a different machine, perhaps in a central control room.

2. System Architecture (Simplified)

This section provides a simplified overview of how the BDF Bakery Production Monitoring System is structured and how data flows through it.

[_Space for diagram: System Architecture Overview_]

2.1. System Layers

The system is organized into three main layers to manage different aspects of its operation:

1. **Entrypoints**: These are the starting points for the application. They launch the main monitoring application and the optional dedicated camera service.

- ``main.py`` starts the main application (default port ``8000``).
- ``camera_main.py`` starts the dedicated camera service (default port ``8001``).

2. **Server Layer:** This layer provides the main interface for operators and other systems. It handles the display of the monitoring UI, live image streaming, AI processing, and color analysis features. It also manages the host-side camera-only API when the system is split.

3. **Library Layer:** This layer contains the core functionalities for the system, including:

- Camera control and image processing.
- Artificial Intelligence (AI) for detection and tracking.
- Color analysis of bakery products.

2.2. Camera Operation Modes

The main application can configure how it interacts with the cameras:

- **Local Mode:** The main application directly communicates with the Basler cameras on the same machine. This is ideal for an all-in-one setup on the production line.
- **Remote Mode:** The main application retrieves images and data from a separate camera service. This allows the camera hardware to be located directly on the production line while the monitoring station can be elsewhere.
- **Emulated Mode:** The system generates synthetic images for testing and development purposes. This means you can test the UI and AI features without requiring live cameras, perfect for development or training.

The dedicated camera service (if used) only supports ``local`` and ``emulated`` modes, focusing purely on image acquisition without AI features.

2.3. Camera Image Pipeline

The process of creating a live image of your bakery products involves several steps:

1. **Camera Discovery:** The system identifies connected Basler cameras.
2. **Camera Setup:** Ensures the correct number of cameras are available and configures settings like triggers and image format.
3. **Image Acquisition:** Reads multiple "strips" or slices from each camera as products move along the line.
4. **Vertical Stacking:** Combines these strips vertically for each camera.
5. **Offset Adjustment:** Applies horizontal and vertical adjustments to align the images precisely.
6. **Image Stitching:** Combines the outputs from both cameras into one seamless, complete image of the bakery product.
7. **Color Conversion:** Converts the stitched image to a standard BGR format if needed, preparing it for display and analysis.

8. **Resizing and Encoding:** Resizes the final image and converts it into a JPEG format for efficient streaming and storage.

9. **Frame Caching:** Stores the latest JPEG image for immediate access by the UI and other services.

Operators can adjust real-time alignment settings using specific API endpoints for optimal image stitching:

- `GET /camera/alignment``: To check current alignment settings.
- `POST /camera/alignment``: To update alignment settings.

[_Space for diagram: Camera Pipeline Flow_]

2.4. Remote Camera Operation

When operating in `remote`` mode, the main application continuously retrieves critical information from the dedicated camera service:

- System and camera status.
- The latest captured image.
- Camera alignment settings.

This setup enables BDF to deploy the camera hardware on the production floor while the main monitoring application, including web and AI capabilities, can be deployed in a separate location, ensuring flexibility and distributed monitoring.

2.5. AI and Color Analysis Pipeline

The AI system is responsible for intelligent inspection of bakery products:

- **Model Selection:** The system uses either RF-DETR for advanced detection and tracking or a combination of YOLO and SORT for general detection and tracking, depending on configuration.
- **Asynchronous Initialization:** AI models start in the background, ensuring the monitoring system itself remains responsive during startup.
- **Status Reporting:** The system provides updates on AI initialization progress via the `/status`` endpoint.
- **Live Stream:** The WebSocket stream initially sends raw images until the AI is ready, then switches to AI-annotated frames.
- **On-Demand Analysis:** The system can perform AI inference on individual images provided via the `POST /inference`` endpoint.

Color Analysis: Integrated within the AI pipeline, a `ColorBuffer`` keeps a history of color data for recently tracked bakery products. This includes:

- Latest color samples for each detected object.
- A rolling history of color changes per object.
- Export options for detailed CSV analysis.
- Production-ready color analysis output for integration with other BDF systems.

2.6. Accessing Production Data

The latest image of a bakery product and its associated analysis data can be accessed through various methods:

- `GET /capture``: To retrieve the latest JPEG image.
- `POST /analyze``: To forward the latest image to external quality analysis services.
- `POST /inference``: To perform local AI analysis on a specific image from disk.
- `WebSocket /ws``: For a live stream of raw or AI-annotated images to operator dashboards.
- The browser UI: Provides a visual interface for monitoring.

2.7. Handling Failures

The BDF Production Monitoring System is designed to be resilient and continues to operate even if certain components encounter issues:

- **Continuous Startup**: The system will attempt to start even if cameras are not immediately available.
- **Status Availability**: The `/status`` endpoint will always be accessible, providing diagnostic information even if image capture fails.
- **Structured Errors**: API endpoints like `/capture``, `/analyze``, and `/inference`` return clear, structured error messages when problems occur, rather than crashing the service.
- **WebSocket Messages**: Operators will receive informative text messages like `warming_up`` or camera error details via the WebSocket stream, rather than a lost connection.

3. System Configuration

This section explains how to configure the BDF Bakery Production Monitoring System to match your operational requirements. The system uses specific files to manage settings and sensitive information.

[_Space for diagram: Configuration Flow_]

3.1. Configuration Sources

The system uses two primary types of configuration files:

- **`settings.json``**: This file stores general operational settings that are not sensitive (e.g., camera modes, stream delays). It can include different profiles for production, debug, or Docker environments.
- **`.env`` and `.env.debug``**: These files store sensitive information (e.g., API keys) and allow for environment-specific overrides. `.env.debug`` can be used for development or testing environments.

The system loads these settings in a specific order, ensuring that environment variables take precedence over `settings.json`` values if a setting is defined in both places. This allows for flexible overrides without altering core files.

3.2. Key Configuration Settings

Below are some important settings BDF operators may need to adjust:

3.2.1. Camera Operating Mode (`CAMERA\_MODE`)

This crucial setting determines how the system interacts with the cameras:

- ``local``: The main system directly controls Basler cameras. Use this when the monitoring system is on the same machine as the cameras.
- ``remote``: The main system retrieves images from a separate camera service. Use this for distributed deployments where the cameras are on a different machine from the monitoring station.
- ``emulated``: The system generates synthetic images. Use this for testing, training, or development without needing actual camera hardware.

When ``CAMERA_MODE`` is set to ``remote``, you'll also need to configure:

- ``REMOTE_CAMERA_BASE_URL``: The network address of the separate camera service (e.g., ``http://<camera-host>:8001``).
- ``REMOTE_CAMERA_TIMEOUT_S``: How long the system waits for a response from the remote camera service.
- ``REMOTE_CAMERA_POLL_INTERVAL_S``: How often the system requests new images from the remote camera service.

When ``CAMERA_MODE`` is set to ``emulated``, you can adjust:

- ``EMULATED_CAMERA_WIDTH`` and ``EMULATED_CAMERA_HEIGHT``: To set the dimensions of the generated images.
- ``EMULATED_CAMERA_FPS``: To set the frame rate of the generated images.
- ``EMULATED_CAMERA_IMAGE_PATH``: An optional setting to use a specific image file instead of generated frames for emulation.

3.2.2. Image Quality and Processing (`CAMERA\_PIXEL\_FORMAT`, `CAMERA\_CONFIG\_BY\_PYLON`)

These settings ensure optimal image quality for bakery product inspection:

- ``CAMERA_PIXEL_FORMAT``: Defines how camera images are interpreted after stitching.
- Use ``BGR8`` or ``BGR8Packed`` if the camera is already configured to output images in the standard OpenCV BGR format.
- Use ``YCbCr422_8`` or similar YUV422 formats if the camera outputs in that format; the system will convert it to BGR for analysis.
- ``CAMERA_CONFIG_BY_PYLON``: When set to ``true``, the application will use the camera settings already configured in Pylon Viewer or another external tool. If ``false``, the application will apply its own settings for pixel format, exposure, and trigger during startup.

3.2.3. AI and Tracking (`DETECTION\_AND\_TRACK`, `AI\_MODEL\_NAME`)

These settings control the AI-driven inspection features:

- `'DETECTION_AND_TRACK'`: Set to `'true'` to enable AI-based detection and tracking of bakery products; set to `'false'` for raw image streaming only.
- `'AI_MODEL_NAME'`: Selects the AI model to be used.
- `'rfdetr'`: Activates the RF-DETR model for advanced detection and tracked annotation.
- Any other value: Activates YOLO detection combined with SORT tracking.

3.2.4. Sensitive Settings (`.env` Files)

The `.env` file is where you store sensitive API keys for external services or Roboflow credentials. Examples include:

```
``env

INFERENCE_API_HEADER=X-API-Key

INFERENCE_API_KEY=your-secret-key-here

RFDETR_MODEL_ID=your-project/your-version

ROBOFLOW_API_KEY=your-roboflow-api-key

``
```

3.3. Recommended Practices for BDF

- **Secrets Management:** Always keep sensitive API keys and credentials in `.env` files, never directly in `settings.json`.
- **Profile Management:** Use `'APP_ENV=debug'` to activate debug settings without changing your production configuration.
- **Docker Configuration:** Use the `'docker'` profile in `settings.json` for settings specific to containerized deployments, especially for network configurations.

4. API Reference for BDF Systems

The API provides a standardized way for BDF's operators and integrated systems to interact with the Racer2 V3 system for production monitoring and data retrieval.

4.1. Service Access Points

- **Main Monitoring Service:** Typically accessed at `'http://localhost:8000'`
- **Dedicated Camera Service (if split):** Typically accessed at `'http://localhost:8001'`

4.2. Key API Endpoints

4.2.1. `'GET /'` - User Interface

- **Purpose:** Loads the operator's web-based monitoring interface.
- **Response:** `'200 text/html'` (the HTML frontend for viewing production).

4.2.2. `GET /status` - System Health Check

- **Purpose:** Provides a quick overview of the system's operational health, including AI initialization, camera readiness, and stream status. Essential for automated monitoring.
- **Example Response:**

```
```json
{
 "ai": {
 "enabled": true,
 "model_name": "rfdetr",
 "phase": "ready",
 "message": "RFDETR ready",
 "error": null,
 "checkpoints": []
 },
 "camera": {
 "ready": true,
 "message": "Camera ready",
 "error": null,
 "backend": "local"
 },
 "stream": {
 "has_frame": true,
 "mode": "tracked"
 }
}
```
```

- **Response:** `200 application/json`

4.2.3. `GET /capture` - Latest Production Image

- **Purpose:** Retrieves the most recent JPEG image of the bakery product from the production line.
- **Success:** `200 image/jpeg` (the image itself).
- **Failure:** `503` if cameras are not ready or no frame is available.

4.2.4. `POST /analyze` - External Quality Check

- **Purpose:** Forwards the latest production image to an external inference service (e.g., a centralized BDF quality analysis system) for further processing. Useful for integrating with broader quality assurance workflows.
- **Behavior:** Sends the image as form data, retries if the external service is temporarily busy (`429`).
- **Success:** `200 application/json` (response from the external service).
- **Failure:** `503` (camera or frame unavailable), `429` (external service busy), or other errors from the external service.

4.2.5. `GET /camera/alignment` - Check Image Stitching

- **Purpose:** Returns the current settings used for stitching images from multiple cameras into a single, seamless product view.
- **Example Response:**

```
```json
{
 "h_offset": 10,
 "v_offset": 190,
 "num_images": 5,
 "backend": "local"
}
```

- **Response:** `200 application/json` or `503` if camera is not initialized.

#### #### 4.2.6. `POST /camera/alignment` - Adjust Image Stitching

- **Purpose:** Allows operators to adjust the horizontal (`h\_offset`), vertical (`v\_offset`), and number of image strips (`num\_images`) used for stitching, ensuring a perfectly aligned view of bakery products.
- **Request Body Example:**

```
```json
```

```
{
  "h_offset": 10,
  "v_offset": 190,
  "num_images": 5
}
...

```

- **Response:** `200 application/json` confirming the new settings and buffer reset.

4.2.7. `GET /color-data` - Retrieve Product Color Analysis

- **Purpose:** Returns the current buffer of color analysis data for tracked bakery products. This data is cleared after being retrieved, making it suitable for one-time data dumps.
- **Response:** `200 application/json` (objects with latest color samples and history). Will be empty if AI or tracking is not enabled, or if no objects have been processed.

4.2.8. `POST /inference` - Local AI Analysis

- **Purpose:** Runs local AI analysis on an image file stored on disk, then saves an annotated version and provides color analysis results. Useful for analyzing specific product samples or previously recorded images.
- **Request Body Example:**

```
```json
{
 "feature_name": "inspection",
 "ImageName": "sample.jpg",
 "SrcDir": "D:/data/job_001",
 "Timestamp": "2026-05-12T13:00:00"
}
...

```

- **Success Response:** Includes the path to the annotated image and color analysis data.
- **Failure:** `404` if the source image is not found, `503` if AI is still loading.

#### #### 4.2.9. `POST /color-analysis` - Current Color Analysis Output

- **Purpose:** Returns the current product-shaped color analysis data and then clears the buffer, similar to `GET /color-data` but with a specific output structure designed for downstream consumption.
- **Response:** `200 application/json` with product color details.

#### #### 4.2.10. `GET /export\_color\_csv` - Export Color Data

- **Purpose:** Exports all buffered color analysis samples as a CSV file, useful for detailed reporting and historical analysis.
- **Response:** `200 text/csv` with `color\_data.csv` attached.

#### #### 4.2.11. `GET /delete\_all\_color\_buffer` - Clear All Color Data

- **Purpose:** Empties the entire buffer of collected color analysis samples.
- **Response:** `200 text/plain`

#### #### 4.2.12. `GET /delete\_color\_buffer/{obj\_id}` - Clear Color Data for One Product

- **Purpose:** Clears color analysis samples specifically for a single tracked product identified by its ID.
- **Response:** `200 text/plain`

#### #### 4.2.13. `WebSocket /ws` - Live Production Stream

- **Purpose:** Provides a continuous stream of either raw JPEG images or AI-annotated JPEG images of the bakery products, enabling real-time monitoring.
- **Behavior:** Sends text messages like `warming\_up` or camera error details when images are not available.

### 4.3. Dedicated Camera Service API

When BDF uses the split deployment option, the dedicated camera service (on port `8001`) provides a simplified set of endpoints for the main application to retrieve camera-specific data:

- `GET /status`: Provides camera status only.
- `GET /camera/alignment`: Returns camera-side image stitching settings.
- `POST /camera/alignment`: Updates camera-side image stitching settings.
- `GET /capture`: Returns the latest raw JPEG image from the camera service.
- `WebSocket /ws`: Streams raw JPEG frames directly from the camera service.

---

## 5. System Deployment

This section outlines the different ways BDF can deploy the Racer2 V3 system, ranging from a simple local setup to a distributed, containerized environment.

[\_Space for diagram: Deployment Options\_]

### 5.1. Initial Setup Checklist

Before any deployment, ensure the following steps are completed:

- **Virtual Environment:** Create and activate a Python virtual environment to manage dependencies.
- **Dependencies:** Install all required software packages from `requirements.txt`.
- **Environment File:** Create your `.env` file from the provided `.env.example` template, filling in BDF-specific values.
- **Settings Review:** Review `settings.json` to confirm the correct configuration profile (e.g., `production`, `debug`, or `docker`).

### 5.2. Deployment Options

#### #### 5.2.1. Option 1: Single-Process Local Deployment (All-in-One)

This is the simplest setup, ideal when the same Windows machine handles both camera access and all monitoring/AI functions.

##### How to Run:

```
""powershell

python .\main.py

""
```

##### Recommended Settings:

- `CAMERA\_MODE=local`
- `DETECTION\_AND\_TRACK=true` or `false`, depending on whether AI monitoring is needed.

##### When to Use:

- Pylon software is installed directly on the machine.
- The machine needs to perform both image acquisition and AI analysis.
- Network separation is not a priority.

#### #### 5.2.2. Option 2: Debug or Hardware-Free Deployment (Testing/Development)

Use this when you need to test the user interface, APIs, or AI workflows without connecting to physical cameras.

##### How to Run:

```
""powershell

$env:APP_ENV="debug"

python .\main.py

""
```

**Recommended Settings:**

- `CAMERA\_MODE=emulated`
- Optionally, `EMULATED\_CAMERA\_IMAGE\_PATH` can be set to use a specific image for consistent testing.

**When to Use:**

- Developing new features or testing configurations without impacting live production.
- Training operators on the UI without requiring camera hardware.

**#### 5.2.3. Option 3: Split Camera and App Deployment (Distributed Monitoring)**

This setup is useful when the camera hardware must remain physically close to the bakery production line, while the web and AI components run on a separate machine (e.g., in a control room).

**Part A: Camera Host**

Run the dedicated camera service on the machine directly connected to the Basler cameras:

```
``powershell
```

```
python .\camera_main.py
```

```
``
```

This service will run on port `8001`.

**Part B: Main App Host**

Run the main application on a separate machine, configured to communicate with the camera host:

**Configuration:**

- Set `CAMERA\_MODE=remote` in `settings.json` or `.env`.
- Set `REMOTE\_CAMERA\_BASE\_URL=http://<camera-host-IP-address>:8001` (replace ``<camera-host-IP-address>`` with the actual IP address or hostname of the camera machine).

**How to Run:**

```
``powershell
```

```
python .\main.py
```

```
``
```

**When to Use:**

- Physical separation of camera hardware from the monitoring station is required.

- To keep the camera processing load off the main AI/UI machine.

#### #### 5.2.4. Option 4: Docker Deployment for the Main App (Containerized Monitoring)

This option uses Docker to run the main application in a container, often in conjunction with a Windows host running the camera service.

##### **Recommended Pattern for BDF:**

- A Windows machine on the production floor runs `camera\_main.py` (as in Option 3, Part A).
- A Docker container (which can be on the same or a different host) runs the main monitoring application.
- The main application inside the Docker container uses `CAMERA\_MODE=remote`.
- Set `REMOTE\_CAMERA\_BASE\_URL=http://host.docker.internal:8001` to allow the Docker container to access the camera service running on the host machine.

##### **How to Run:**

```
""powershell
```

```
docker compose up --build
```

```
""
```

##### **When to Use:**

- To leverage containerization benefits (isolation, portability) for the main application.
- For consistent deployment across different environments.

#### 5.3. Environment Considerations for BDF

- **Local Camera Mode:** Ensure Pylon software is installed and the application has permission to access both Basler cameras directly.
- **Remote Camera Mode:** Verify stable network connectivity between the main app host and the camera host. The camera host must expose its API endpoints (`/status`, `/capture`, `/camera/alignment`).
- **Emulated Mode:** No camera hardware is required, making it the safest option for front-end development and integration testing.

#### 5.4. Verification Checklist Post-Deployment

After any deployment, perform these checks to ensure the BDF Production Monitoring System is operating correctly:

1. **System Status:** Access `GET /status` to confirm it returns a `200` response.
2. **Camera Readiness:** Verify that `camera.ready` is `true` in the `/status` response, indicating the chosen camera mode is active.
3. **Image Capture:** Confirm that `GET /capture` successfully returns a JPEG image.

4. **Browser Interface:** Open the browser UI (typically `http://localhost:8000`) and verify it loads and displays live images.
5. **Live Stream:** Check that WebSocket streaming is functional in the browser UI or via a WebSocket client.
6. **AI Status (if enabled):** Wait for `ai.phase` to become `ready` in the `/status` response before testing AI-specific features.
7. **Remote Service Health (if used):** If using remote mode, verify that the dedicated camera service on port `8001` also reports a healthy `/status`.

#### 5.5. Operational Notes for BDF

- The system is designed to be resilient; it will remain online and provide diagnostic information even if there are initial camera startup issues.
- AI model initialization may take longer than the general server startup. This is normal, and `ai.phase` in the `/status` endpoint will show progress.
- `POST /inference` requires direct access to image files on the machine where the main application is running.

---

## 6. Troubleshooting

This section provides guidance for BDF operators on how to diagnose and resolve common issues with the Racer2 V3 Bakery Production Monitoring System.

### 6.1. Server Starts But No Images Arrive

#### Symptoms:

- The monitoring interface is blank or static.
- `GET /status` shows `camera.ready` as `false`.
- The WebSocket stream receives text messages like `warming_up`.

#### What to Check:

- **System Status:** Always start by checking `GET /status`.
- **Local Mode:**
  - Verify that Pylon software is correctly installed on the machine.
  - Ensure both Basler cameras are physically connected and powered on.
  - Confirm that the application has the necessary permissions to access the cameras.
- **Remote Mode:**
  - Check the `REMOTE_CAMERA_BASE_URL` setting for any typos.
  - Verify that the dedicated camera service is running on the camera host (see Option 3 in Deployment).
  - Test `GET <REMOTE_CAMERA_BASE_URL>/capture` directly on the camera host to ensure it returns a JPEG image.
- **Emulated Mode:**



- Confirm that `CAMERA\_MODE=emulated` is correctly set.
- If `EMULATED\_CAMERA\_IMAGE\_PATH` is configured, ensure the specified image file exists and is readable.

## 6.2. Camera Busy Or Unavailable

### Symptoms:

- `GET /status` reports that the camera is not ready.
- System logs mention camera initialization failures or hardware being busy.

### What to Check:

- **Close Other Applications:** Ensure no other programs, such as Pylon Viewer, are currently using the cameras.
- **Previous Instances:** Confirm that no older versions or duplicate copies of the Racer2 V3 service are still running in the background.
- **Restart Host:** If the camera driver appears unresponsive, try restarting the camera host machine.

## 6.3. Remote Camera Mode Does Not Update Images

### Symptoms:

- Images from the remote camera service are not refreshing or are outdated.

### What to Check:

- **URL Accuracy:** Double-check `REMOTE\_CAMERA\_BASE\_URL` for any typos.
- **Port Access:** Verify that port `8001` on the camera host is accessible from the main application's machine (e.g., not blocked by a firewall).
- **Remote Status:** Confirm that `GET <REMOTE\_CAMERA\_BASE\_URL>/status` returns a `200` response from the camera host.
- **Network Reachability:** Ensure the main application can establish a network connection to the camera host.
- **Common Causes:**
  - Docker container network issues preventing access to the host URL.
  - Windows firewall blocking the camera service on the camera host.
  - The camera service is running, but `GET /capture` on the camera host is failing.

## 6.4. WebSocket Stream Shows Text Instead Of Images

### Symptoms:

- The live stream displays messages like `warming\_up` or error text instead of bakery product images.

### Explanation:

This behavior is normal and expected when:

- The camera system is still starting up or warming up.
- Camera initialization has failed.
- In remote mode, the system has not yet successfully retrieved a valid image frame from the camera service.

The text messages provide status updates to the operator.

### 6.5. AI Never Becomes Ready

#### Symptoms:

- `GET /status` shows `ai.phase` as `pending` or `active` for a prolonged period, but never `ready`.

#### What to Check:

- **System Status:** Check `GET /status` for detailed AI checkpoint progress and error messages.
- **Credentials:** Verify that `.env` contains valid Roboflow API keys (`ROBOFLOW_API_KEY`) and RF-DETR model IDs (`RFDETR_MODEL_ID`) if applicable.
- **Model Selection:** Confirm that the `AI_MODEL_NAME` in your configuration is set to the intended model (e.g., `rfdetr` or another value for YOLO/SORT).
- **Local Environment:** Ensure the local runtime environment (drivers, hardware) is compatible with the selected AI model.

### 6.6. `POST /inference` Returns 404

#### Symptoms:

- When attempting to run local AI analysis on an image, the API returns a `404 Not Found` error.

#### What to Check:

- **Source Directory:** Verify that `SrcDir` in your request points to a valid and accessible directory on the machine running the main application.
- **Image Name:** Ensure `ImageName` is spelled correctly and the file actually exists within `SrcDir`.
- **Combined Path:** Confirm that the full path `SrcDir/ImageName` leads to an existing file.

**Important Note:** This endpoint reads image files directly from the local disk of the machine running the main application; it cannot fetch images from remote network locations.

### 6.7. Color Endpoints Return Empty Data

#### Symptoms:

- `GET /color-data` or `POST /color-analysis` returns an empty list of objects or no data.

#### Possible Reasons:

- **Detection/Tracking Disabled:** `DETECTION_AND_TRACK` might be set to `false` in your configuration.

- **AI Not Ready:** The AI model may still be loading or has not yet reached `ready` phase.
- **No Tracked Objects:** No bakery products have been detected and tracked by the AI since the last buffer clear or system startup.
- **Buffer Already Cleared:** A previous call to `GET /color-data` or `POST /color-analysis` already retrieved and cleared the buffer, or `GET /delete\_all\_color\_buffer` was called.

## 6.8. External Inference Endpoint Fails

### Symptoms:

- `POST /analyze` fails to send images to an external service or receives errors.

### What to Verify:

- **URL Accuracy:** Check `INFERENCE\_URL` in `settings.json` or `.env` for typos.
- **API Header/Key:** Verify `INFERENCE\_API\_HEADER` and `INFERENCE\_API\_KEY` in `.env` are correct.
- **Service Availability:** Ensure the external inference service is running and accessible from the machine running the main application.
- **Network Reachability:** Confirm that the main application can establish a network connection to the external inference service.

## 6.9. Pixel Format Problems

### Symptoms:

- Output images show incorrect colors.
- Error messages appear during image capture or conversion related to YUV.
- OpenCV errors mentioning YUV conversion.

### What to Verify:

- **Pixel Format Setting:** Ensure `CAMERA\_PIXEL\_FORMAT` in your configuration exactly matches the actual pixel format output by your Basler cameras.
- **Pylon Configuration:** If you've configured camera features externally, set `CAMERA\_CONFIG\_BY\_PYLON=true` to prevent the application from overwriting those settings.
- **BGR8 Usage:** Use `BGR8` or `BGR8Packed` when your camera is already set to output images in the OpenCV-ready BGR format. YUV formats require conversion within the application.

## 6.10. General Diagnostic Sequence for BDF Operators

When encountering an issue, follow this systematic approach:

1. **Check System Status:** Call `GET /status`.
2. **Confirm Camera Readiness:** Verify `camera.ready` is `true`.
3. **Test Image Capture:** Check if `GET /capture` successfully returns a JPEG image.

4. **Verify UI Update:** Confirm that the browser monitoring page on `/` loads and refreshes correctly.

5. **AI Readiness (if enabled):** If AI is part of your workflow, wait for `ai.phase=ready` in the `/status` response.

6. **Test AI/Color Features:** Once AI is ready, then proceed to test `/analyze`, `/inference`, `GET /color-data`, or WebSocket streaming.

---

## 7. Core Software Components (Libraries)

The Racer2 V3 system relies on several internal software components, or libraries, which provide the core functionalities for camera control, AI, and color analysis. While operators typically interact with the system through the main application, understanding the role of these components can provide a clearer picture of the system's capabilities for BDF.

### 7.1. Camera Library

- **Purpose:** This component is responsible for all camera-related operations, including finding Basler cameras, configuring their settings, acquiring image slices, stitching them together into a complete image of the bakery product, and preparing these images as JPEGs for the rest of the system.
- **Key Functions:** Handles line-scan acquisition, vertical stacking of image slices, horizontal stitching across multiple cameras, and preparing images for display and analysis.
- **Used By:** The local camera runtime and the dedicated camera service.

### 7.2. AI Library

- **Purpose:** This component manages all Artificial Intelligence processes. It includes selecting the appropriate AI model, initializing it in the background, performing real-time inference on live images from the production line, analyzing individual images from files, and adding visual annotations (like bounding boxes) to highlight detections. It also integrates optional tracking features.
- **Key Functions:** AI model selection, asynchronous model initialization, live and single-image inference, annotation overlay, and optional object tracking.
- **Used By:** The main application for AI-assisted inspection.

### 7.3. Color Library

- **Purpose:** This component specializes in the color analysis of detected bakery products. It extracts color information around tracked objects, maintains a history of color samples for each product, and provides tools for converting color data into various formats (e.g., RGB, CIELAB) for detailed quality control and reporting.
- **Key Functions:** Per-object color extraction, rolling history buffering for color samples, and color conversion utilities.
- **Used By:** The AI pipeline to enrich detection and tracking with valuable color data.